

PLAYWRIGHT BASICS

Locating Elements in Playwright: CSS and XPath Selectors

Selector Type	Example Code	Description
CSS (tag)	<code>`page.locator('button')`</code>	Selects all <code><button></code>
CSS (class)	<code>`page.locator('.item-class')`</code>	Selects elements by class
CSS (id)	<code>`page.locator('#username')`</code>	Selects element by id
CSS (attr)	<code>`page.locator('a[href="/login"]')`</code>	Selects by attribute
XPath	<code>`page.locator('xpath=//button[text()="Submit"]')`</code>	Button with text 'Submit'
XPath (short)	<code>`page.locator('//div[@class="item-class"]')`</code>	Div with class 'item-class'

Example Usage:

```
``typescript
await page.locator('input[name="email"]').fill('user@example.com');
await page.locator('xpath=//button[contains(text(),"Login")]').click();
````
```

## ## Playwright Built-in Locator Examples

| Locator Type | Example Code                                                | Description                 |
|--------------|-------------------------------------------------------------|-----------------------------|
| CSS Selector | <code>`page.locator('button.primary')`</code>               | Button with class 'primary' |
| Text         | <code>`page.getByText('Sign In')`</code>                    | By visible text             |
| Role         | <code>`page.getByRole('button', { name: 'Submit' })`</code> | By ARIA role & name         |
| Label        | <code>`page.getByLabel('Email')`</code>                     | By label text               |
| Placeholder  | <code>`page.getByPlaceholder('Search')`</code>              | By placeholder text         |
| Alt Text     | <code>`page.getByAltText('Logo')`</code>                    | By image alt text           |
| Title        | <code>`page.getByTitle('Close')`</code>                     | By title attribute          |
| Test ID      | <code>`page.getByTestId('cart-icon')`</code>                | By data-testid attribute    |
| XPath        | <code>`page.locator('xpath=//div[@id="main"]')`</code>      | By XPath                    |

### ## CHAINING SELECTORS:

| Scenario | Example Code | Description |
|----------|--------------|-------------|
|          |              |             |

|                         |                                                                            |                                  |  |
|-------------------------|----------------------------------------------------------------------------|----------------------------------|--|
| Button in card          | `page.locator('.product-card').getByRole('button', { name: 'Add' })`.`     | 'Add' button inside product card |  |
| Input in form           | `page.locator('#login-form').locator('input[type="email"]')`               | Email input inside login form    |  |
| Span in product row     | `page.locator('tr[data-product-id="123"]').locator('span.price')`          | Price span inside product row    |  |
| Checkbox in label       | `page.getByLabel('Accept Terms').locator('input[type="checkbox"]')`        | Checkbox inside label            |  |
| Button in visible modal | `page.locator('.modal:visible').getByRole('button', { name: 'Checkout' })` | 'Checkout' button in modal       |  |
| Span in active nav link | `page.locator('nav').locator('a.active').locator('span')`                  | Span inside active nav link      |  |
| -----                   |                                                                            |                                  |  |

```typescript

// Find a button with text 'Checkout' inside a visible modal dialog

await page.locator('.modal:visible').getByRole('button', { name: 'Checkout' }).click();

// Find a link inside a navigation bar, then a span inside that link

await page.locator('nav').locator('a.active').locator('span').textContent();

```

## ## FILTERING LOCATORS:

```typescript

// Filter by text content

await page.locator('button', { hasText: 'Save' }).click();

// By NOT having text

await page.locator('button').filter({ hasNotText: 'Cancel' })

// Filter by another element inside (child/descendant)(e.g., a span with text)

await page.locator('div.card', { has: page.locator('span.price') }).click();

// Filter by visible state

await page.locator('button:visible').click();

// Filter by nth occurrence (e.g., second button)

await page.locator('button').nth(1).click();

// Filter by attribute value

await page.locator('input[type="checkbox"]:checked').first().uncheck();

// Filter by role and name

await page.getByRole('button', { name: 'Delete', exact: true }).click();

```

## ## Locator Strategy Best Practices

-  **\*\*Prefer built-in locators:\*\*** Use `getByRole`, `getByLabel`, `getByText` for robust selectors.

-  **\*\*Use stable attributes:\*\*** Use ``data-testid`` or similar for long-term stability.
-  **\*\*Avoid brittle selectors:\*\*** Don't use auto-generated classes or deep CSS/XPath.
-  **\*\*Be specific but not overly strict:\*\*** Uniquely identify elements, but allow for minor UI changes.
-  **\*\*Chain locators for context:\*\*** Use ``locator().locator()`` to scope within parents.
-  **\*\*Use text and role for accessibility:\*\*** Prefer roles and accessible names.
-  **\*\*Avoid dynamic text:\*\*** If text changes often, use a stable attribute instead.
-  **\*\*Use filters for precision:\*\*** Use ``{ hasText: '...' }``, ``{ has: ... }``, or ``filter()``.
-  **\*\*Keep selectors DRY:\*\*** Store common selectors in page objects or constants.
-  **\*\*Validate selectors manually:\*\*** Test selectors in codegen or browser dev tools.

#### **\*\*Summary:\*\***

Choose selectors that are stable, readable, and user-centric for reliable, maintainable tests.

#### **## Here are Playwright examples for handling INPUT BOXES, BUTTONS, and CHECKBOXES:**

```typescript

INPUT BOX

// Fill an input box with text

```
await page.locator('input[name="email"]').fill('user@example.com');
```

// Clear and type into an input box

```
await page.locator('#username').clear();
```

```
await page.locator('#username').type('myUser');
```

BUTTONS

// Click a button by text

```
await page.getByText('Submit').click();
```

// Click a button by role and name

```
await page.getByRole('button', { name: 'Login' }).click();
```

// Click a button by CSS selector

```
await page.locator('.btn-primary').click();
```

CHECKBOXES

// Check a checkbox

```
await page.locator('input[type="checkbox"][name="accept"]').check();
```

// Uncheck a checkbox

```
await page.locator('input[type="checkbox"][name="subscribe"]').uncheck();
```

// Toggle a checkbox (click)

```
await page.locator('#newsletter-checkbox').click();
```

SUMMARY

Use `.fill()` or `.type()` for input boxes.

Use `.click()` for buttons.

Use `.check()`, `.unchecked()`, or `.click()` for checkboxes.

Use locators that best match your element (CSS, text, role, etc.).

...

HOW TO CHECK MULTI CHECKBOXES

```typescript

// Check all checkboxes with a common selector

const checkboxes = page.locator('input[type="checkbox"]');

const count = await checkboxes.count();

for (let i = 0; i < count; i++) {

await checkboxes.nth(i).check();

}

// Uncheck All Checkboxes

const checkboxes = page.locator('input[type="checkbox"]');

const count = await checkboxes.count();

for (let i = 0; i < count; i++) {

await checkboxes.nth(i).unchecked();

}

// Check Specific Checkboxes by Value

await page.locator('input[type="checkbox"][value="option1"]').check();

await page.locator('input[type="checkbox"][value="option3"]').check();

// Check All Visible Checkboxes

const checkboxes = page.locator('input[type="checkbox"]:visible');

const count = await checkboxes.count();

for (let i = 0; i < count; i++) {

await checkboxes.nth(i).check();

}

Summary:

Use `.locator('input[type="checkbox"]')` to select all checkboxes.

Use `.nth(i)` to loop through and check/uncheck each one.

Use attribute selectors to target specific checkboxes.

```
```

```typescript

How to handle DROP-DOWN element and MULTI-SELECT DROP-DOWN elements

Single-Select Drop-down
// Select by visible text
await page.locator('select#country').selectOption({ label: 'India' });

// Select by value
await page.locator('select#country').selectOption('IN');

// Select by index
await page.locator('select#country').selectOption({ index: 2 });

Multi-Select Drop-down
// Select multiple options by value
await page.locator('select#fruits').selectOption(['apple', 'banana']);

// Select multiple options by label
await page.locator('select#fruits').selectOption([{ label: 'Apple' }, { label: 'Banana' }]);

Get Selected Options

const selected = await page.locator('select#fruits').evaluate((el) =>
 Array.from(el.selectedOptions).map(option => option.value)
);
console.log(selected); // ['apple', 'banana']

Summary:

Use .selectOption() for both single and multi-select drop-downs.
Pass a string, object, or array for multiple selections.
Use .evaluate() to get selected values if needed.

```

## How to handle RADIO BUTTONS and MOUSE DRAG and DROP
```typescript

RADIO BUTTON

// Select a radio button by value
```



```
await page.locator('input[type="radio"][value="male"]').check();

// Select a radio button by label text
await page.getByLabel('Female').check();

// Verify if a radio button is checked
const isChecked = await page.locator('input[type="radio"][value="male"]').isChecked();
console.log(isChecked); // true or false
```

#### MOUSE DRAG and DROP

-----

```
// Drag an element and drop it onto another element
const source = page.locator('#drag-source');
const target = page.locator('#drop-target');
await source.dragTo(target);

// Drag by offset (e.g., move 100px right and 50px down)
await page.locator('#drag-source').dragTo(page.locator('#drag-source'), {
 targetPosition: { x: 100, y: 50 }
});
```

Summary:

Use `.check()` to select radio buttons.  
Use `.dragTo()` for drag-and-drop actions between elements or by offset.  
...

#### ## How to handle UPLOAD and DOWNLOAD a file show an example

```typescript

FILE UPLOAD

```
// Upload a file to an <input type="file">
await page.setInputFiles('input[type="file"]', 'path/to/file.txt');

// Upload multiple files
await page.setInputFiles('input[type="file"]', [
  'path/to/file1.txt',
  'path/to/file2.jpg'
]);
```

FILE DOWNLOAD

```
=====
```

SINGLE FILE DOWNLOAD

```
// Start waiting for the download before clicking
const [ download ] = await Promise.all([
  page.waitForEvent('download'), page.locator('a#download-link').click(), // or any action that triggers download
]);

// Save the downloaded file to a specific path
await download.saveAs('downloads/myfile.txt');
```

MULTI-FILE DOWNLOAD

```
-----
```

```
// Start waiting for multiple downloads
const downloadPromises = [
  page.waitForEvent('download'),
  page.waitForEvent('download')
];

// Trigger multiple downloads (e.g., clicking two links)
await Promise.all([
  page.locator('a#download-link1').click(),
  page.locator('a#download-link2').click()
]);

// Wait for both downloads to complete
const [download1, download2] = await Promise.all(downloadPromises);

// Save the files
await download1.saveAs('downloads/file1.txt');
await download2.saveAs('downloads/file2.txt');
```

Summary:

```
-----
```

Use `page.setInputFiles()` for file uploads.
Use `page.waitForEvent('download')` and `download.saveAs()` for file downloads.
For a single file, use `page.waitForEvent('download')` and `download.saveAs()`.
For multiple files, set up multiple `waitForEvent('download')` promises and trigger all downloads, then save each file.

PROGRAMMING EXAMPLE

```
-----
```

```
import { test, expect } from '@playwright/test';
```

```
import * as fs from 'fs';

test('Single file download', async ({ page, tmpPath }) => {
  // Go to the page with the download link
  await page.goto('https://example.com/downloads');

  // Wait for the download to start after clicking the download link/button
  const [download] = await Promise.all([
    page.waitForEvent('download'),
    page.locator('a#download-link').click(), // Replace with your actual selector
  ]);

  // Save the downloaded file to a specific path
  const filePath = `${tmpPath}/myfile.txt`;
  await download.saveAs(filePath);

  // Optionally, verify the file exists
  expect(fs.existsSync(filePath)).toBeTruthy();
});

test('Multi-file download', async ({ page, tmpPath }) => {
  // Go to the page with multiple download links
  await page.goto('https://example.com/downloads');

  // Prepare to wait for multiple downloads
  const downloadPromise1 = page.waitForEvent('download');
  const downloadPromise2 = page.waitForEvent('download');

  // Trigger both downloads (e.g., clicking two links/buttons)
  await Promise.all([
    page.locator('a#download-link1').click(), // Replace with your actual selector
    page.locator('a#download-link2').click()
  ]);

  // Wait for both downloads to complete
  const download1 = await downloadPromise1;
  const download2 = await downloadPromise2;

  // Save the files
  const filePath1 = `${tmpPath}/file1.txt`;
  const filePath2 = `${tmpPath}/file2.txt`;
  await download1.saveAs(filePath1);
  await download2.saveAs(filePath2);

  // Optionally, verify the files exist
});
```



```
    expect(fs.existsSync(filePath1)).toBeTruthy();
    expect(fs.existsSync(filePath2)).toBeTruthy();
  });
  ...

```

iFrames Topic:

```typescript

### SWITCHING TO AN IFRAME AND INTERACT

-----

```
// Locate the iframe element
const frameLocator = page.frameLocator('iframe#my-frame'); // Use CSS selector for the iframe

// Interact with elements inside the iframe
await frameLocator.locator('input[name="username"]').fill('myUser');
await frameLocator.locator('button[type="submit"]').click();

```

### GET A FRAME BY NAME OR URL

-----

```
// Get the frame object by name or URL
const frame = page.frame({ name: 'my-frame' }); // or use { url: /partial-url/ }
// Here (?) is for optional chaining, if the frame is present then it will fill or else it will continue with next step
await frame?.locator('input[name="username"]').fill('myUser');

```

### LIST ALL FRAMES

-----

```
const frames = page.frames();
frames.forEach(f => console.log(f.url()));

```

Summary:

-----  
Use `page.frameLocator('iframe-selector')` for direct, chainable access to elements inside an iframe.  
Use `page.frame({ name, url })` to get a frame object and interact `with` it.  
Always use the correct selector or frame name to target the right iframe.

### NESTED IFRAMES –

-----

A nested iframe is an iframe that is placed inside another iframe on a web page. This means you have a parent iframe, and inside its `HTML`, there is `another` (child) iframe.

Example `HTML` structure –

```

<!-- Main page -->
<iframe id="frameA" src="a.html"></iframe>

<!-- a.html -->
<html>
 <body>
 <iframe id="frameB" src="b.html"></iframe>
 </body>
</html>

<!-- b.html -->
<html>
 <body>
 <iframe id="frameC" src="c.html"></iframe>
 </body>
</html>

<!-- c.html -->
<html>
 <body>
 <input id="deepInput" type="text" />
 </body>
</html>
```

Playwright Code to Access the Deeply Nested Input -

```

// Get the first (outermost) iframe
const frameA = page.frame({ name: 'frameA' });

// Get the second iframe inside frameA
const frameB = frameA?.frame({ name: 'frameB' });

// Get the third iframe inside frameB
const frameC = frameB?.frame({ name: 'frameC' });

// Interact with the input inside frameC
await frameC?.locator('#deepInput').fill('Hello from Playwright!');
...
```

# How to handle JAVASCRIPT ALERTS:

=====

```
```typescript
```

In Playwright, handling window tabs (new browser tabs or windows opened via links, JavaScript, or user actions) involves managing multiple Page instances within a BrowserContext. Here's how to work with them:

The key reason is that Playwright treats NEW TABS and POP-UP WINDOWS almost identically. Both are considered NEW "PAGES" that are created within the existing browser context.

The core mechanism you need to use is the same for both scenarios:

Listen for the popup event: The `context.waitForEvent('popup')` command is the universal way to capture any new page that is created, regardless of whether it appears as a new tab or a separate, smaller browser window.

Capture the New Page Object: The event listener returns a Promise that resolves with the Page object of the NEW TAB or WINDOW.

Interact with the New Page: Once you have that Page object, you can interact with it just like any other page—you can find elements, click buttons, fill forms, and run assertions.

EXAMPLE 1 –

=====

```
/**
 * This test suite demonstrates advanced handling of multiple tabs and windows in Playwright.
 *
 * The test case covers the following key scenarios:
 * 1. **Initial Navigation**: Starts by navigating to the Playwright documentation page.
 * 2. **Handling a New Tab from a Click**: It clicks a link that opens a new tab (the GitHub repository)
 *    and correctly captures the new page object using `context.waitForEvent('popup')`.
 * 3. **Opening a Tab Manually**: It programmatically opens a third tab using `context.newPage()` and
 *    navigates it to Google.
 * 4. **Managing and Switching Between Tabs**: It retrieves a list of all open pages using `context.pages()`
 *    and demonstrates how to switch focus between them using `bringToFront()`.
 * 5. **Closing Tabs**: It closes the two extra tabs that were opened.
 * 6. **Final State Verification**: It confirms that only the original page remains open at the end of the test.
 */
import { test, expect, type Page, type BrowserContext } from 'playwright/test';

test.describe('Advanced Multi-Tab/Window Handling', () => {

  test('should seamlessly switch between multiple tabs and windows', async ({ page, context }) => {

    await page.goto('https://playwright.dev/');
    await expect(page).toHaveTitle(/Playwright/);
    console.log(`1. Initial page: ${page.url()}`);
```

```

const newTabPromise = context.waitForEvent('popup');
await page.getByRole('link', { name: 'GitHub repository' }).click();
const githubPage = await newTabPromise;
await githubPage.waitForLoadState();

console.log(`2. New tab opened: ${githubPage.url()}`);
await expect(githubPage).toHaveTitle(/microsoft\/playwright/);

const googlePage = await context.newPage();
await googlePage.goto('https://www.google.com');
await googlePage.waitForLoadState();

console.log(`3. Second new tab opened: ${googlePage.url()}`);
await expect(googlePage).toHaveTitle(/Google/);

const allPages = context.pages();
console.log(`4. Total pages open: ${allPages.length}`);
expect(allPages.length).toBe(3);

await page.bringToFront();
console.log(` - Switched back to: ${page.url()}`);
await expect(page.getByRole('heading', { name: 'Playwright enables reliable end-to-end testing for modern web apps.' })).toBeVisible();

await githubPage.bringToFront();
console.log(` - Switched to: ${githubPage.url()}`);
await expect(githubPage.getByRole('link', { name: 'playwright', exact: true })).toBeVisible();

await googlePage.bringToFront();
console.log(` - Switched to: ${googlePage.url()}`);
await expect(googlePage.getByRole('img', { name: 'Google' })).toBeVisible();

await googlePage.close();
console.log('5. Closed Google page.');

await githubPage.close();
console.log(' - Closed GitHub page.');

console.log(`6. Current page is now: ${page.url()}`);
await expect(page).toHaveTitle(/Playwright/);
expect(context.pages().length).toBe(1);
});
});

```

```
=====
```

```
/**
 * This test demonstrates how to handle a pop-up window in Playwright.
 *
 * The test performs the following actions:
 * 1. Navigates to a page that contains a button designed to open a pop-up window.
 * 2. Sets up a listener (`context.waitForEvent('popup')`) to capture the pop-up before it's opened.
 * 3. Clicks the button to trigger the pop-up.
 * 4. Waits for the pop-up window to load and then interacts with it by verifying its title and content.
 * 5. Closes the pop-up window.
 * 6. Returns focus to the main page and verifies its title to ensure the test can continue.
 */
```

```
import { test, expect } from 'playwright/test';
```

```
test('should handle a pop-up window', async ({ page, context }) => {
  await page.goto('https://example.com/with-a-popup');

  const popupPromise = context.waitForEvent('popup');

  await page.getByRole('button', { name: 'Open Help Window' }).click();

  const popupWindow = await popupPromise;
  await popupWindow.waitForLoadState();

  await expect(popupWindow).toHaveTitle('Help Center');
  const helpText = await popupWindow.getByText('This is our help documentation. ');
  await expect(helpText).toBeVisible();

  await popupWindow.close();

  await expect(page).toHaveTitle('Main Application');
});
```

```
...
```

How to handle SHADOW DOM in Playwright

```
```typescript
```

What is Shadow DOM?

First, a quick overview: Shadow DOM is a web standard that allows a developer to encapsulate a portion of a web page's structure and styling. This is a core part of Web Components. Elements inside a "shadow root" are hidden from the main document's DOM, which means they can't be selected with standard query selectors like `document.querySelector()`.

This encapsulation can make them tricky to test with other automation tools, but not with Playwright.



## How Playwright Handles Shadow DOM

Playwright's locators are built to automatically pierce the Shadow DOM. This means you can write your selectors as if the shadow boundaries don't exist. You don't need any special commands or syntax.

EXAMPLE : INTERACTING WITH A SHADOW DOM FORM -

=====

Let's imagine a web page has a custom web component called `<user-card>` that uses a shadow root to hide its internal structure.

```
<body>
 <h1>Welcome!</h1>

 <user-card>
 <!-- The following is hidden inside the shadow root -->
 #shadow-root (open)
 <div class="card">
 <p>Username: test-user</p>
 <button id="details-btn">Show Details</button>
 <input type="text" placeholder="Enter new username">
 </div>
 </user-card>

</body>
```

```
import { test, expect } from '@playwright/test';
import path from 'path';

/**
 * This test demonstrates a real-world scenario for testing a custom web component
 * with a Shadow DOM. It navigates to a local HTML file containing the <user-card>
 * component and interacts with elements inside its Shadow DOM.
 *
 * Test Actions:
 * 1. Constructs the file path and URL for the local HTML page.
 * 2. Navigates the browser to the specified file URL.
 * 3. Locates the 'Show Details' button and the username input field within the
 * <user-card> component's Shadow DOM.
 * 4. Asserts that the button is visible and contains the correct text.
 * 5. Fills the username input with a new value ('updated-user').
 * 6. Asserts that the input field's value has been successfully updated.
 */
test.describe('Live Shadow DOM Test for <user-card>', () => {
```

```
test('should interact with the live user-card component', async ({ page }) => {
 const htmlFilePath = path.join(process.cwd(), 'test-pages', 'user-card.html');
 const fileUrl = `file://${htmlFilePath}`;

 await page.goto(fileUrl);

 const detailsButton = page.locator('user-card #details-btn');
 const usernameInput = page.locator('user-card input[type="text"]');

 await expect(detailsButton).toBeVisible();
 await expect(detailsButton).toHaveText('Show Details');

 const newUsername = 'updated-user';
 await usernameInput.fill(newUsername);

 await expect(usernameInput).toHaveValue(newUsername);
});

});

...

```